Ders 23 Çalışma Özeti

Ders 23 Çalışma Özeti

Genel Konular

- Dizgi (string) sıralamanın sayı sıralamasından farkı.
- Anahtar indeksli sayma (key-indexed counting): her karakterin sıralı dizide nereden başlayacağını sayarak bulma.
- LSD dizgi sıralama (least significant digit first string sort): sabit uzunluklu dizgileri sağdan sola karakter karakter sıralama.
- Kararlı sıralama (stable sorting) kavramı.
- Yedek dizi kullanmanın bellek açısından değerlendirmesi.
- Vize sınavı sorularının çözümü ve sık yapılan hatalar.
- MSD (most significant digit first) dizgi sıralama öğrencilere inceleme için bırakıldı.

Hocanın Özellikle Vurguladığı Kısımlar

- Sayılar bir bütün olarak karşılaştırılır (34 ile 45 gibi). Dizgilerde her karakter ayrı bir değerdir; karşılaştırma karakter karakter yapılır. İlk farklı karakterde karar verilir.
- C'de karakterler sayı gibi kullanılabilir: `'a' - 'a' = 0`, `'c' - 'a' = 2`. Bu fark doğrudan dizi indisi olarak kullanılır.
- `count` dizisi mutlaka sıfırlanmalıdır. Sıfırlanmazsa bellekte kalan değerlerin üzerine eklenir.
- LSD sıralama kararlıdır: aynı değere sahip iki elemanın önceki adımdaki göreli sırası korunur. Korunmazsa önceki adımlarda yapılan sıralama boşa gider.
- Yedek dizi kullanan algoritmalar (merge sort, key-indexed counting) zaman kazandırır ama bellek harcar. Eleman sayısı çok büyükse bu bir sorun olabilir. Selection sort, insertion sort, quicksort gibi algoritmalar bu yüzden dizinin kendisi üzerinde çalışır.
- İkinci sınıfın sonunda yalnız "çalışan" kod yeterli değildir. Tasarım da doğru olmalıdır. Aynı listeyi gereksiz yere birden fazla kez gezmek verimsiz bir tasarımdır.
- Veri yapıları ve algoritmalar temel bir derstir. Hoca yaz tatilinde bol bol farklı algoritma problemi çözmeyi önerdi.

Kısa Tekrar Notları

- Basit sıralamalar (selection, bubble, insertion): en kötü durum $O(n^2)$.
- Merge sort ve heap sort: en kötü durum $O(n \log n)$.
- Quicksort: en kötü durum $O(n^2)$ (sıralı girdi), ortalama $O(n \log n)$. Dizi önce karıştırılırsa (shuffle) en kötü durumdan kaçınılır.
- Key-indexed counting adımları:
  - Frekansları say: `count[a[i] - 'a' + 1]++`.
  - Frekansları başlangıç indislerine çevir: `count[r + 1] += count[r]`.
  - Elemanları `tmp` dizisine dağıt: `tmp[count[a[i] - 'a']++] = a[i]`.
  - `tmp` dizisini `a` dizisine geri kopyala.
- LSD sıralama: W karakterli dizgilerde son karakterden ilk karaktere kadar her konum için key-indexed counting uygulanır.
- Kararlı sıralama: eşit anahtarlı elemanların göreli sırası korunur.

Detaylı Açıklamalar

Dizgi Karşılaştırma

- `kalem` ile `kel` karşılaştırılsın. İlk karakterler (`k`) aynı. İkinci karakterde `a < e` olduğu için `kalem < kel` olur.
- Sözlükteki sıralama mantığı aynen budur.
- `kaz` ile `kaza` da karakter karakter karşılaştırılır; bütün olarak karşılaştırılamaz.

Anahtar İndeksli Sayma (Key-Indexed Counting)

Varsayım: anahtarlar 0 ile $R-1$ arasında tam sayılardır ve dizi indisi olarak kullanılabilir. Burada R sembol sayısıdır (örnekte `a-f`, yani $R = 6$).

Örnek dizi (12 eleman):

```
indis:  0  1  2  3  4  5  6  7  8  9  10 11
a:      d  a  c  f  f  b  d  b  f  b  e  a
```

1. Frekansları sayma. Her karakterin sayısı bir sonraki göze yazılır. Böylece bir sonraki adımda başlangıç indisleri kolayca bulunur.

Karakter	a	b	c	d	e	f
Sayı	2	3	1	2	1	3

2. Başlangıç indislerini bulma. Frekanslar kümülatif toplanır.

Karakter	a	b	c	d	e	f	(son)
Başlangıç	0	2	5	6	8	9	12

- `a` 0'dan başlar. İki `a` olduğu için `b` 2'den başlar. Üç `b` olduğu için `c` 5'ten başlar.

3. Dağıtma. Dizi baştan sona gezilir. Her karakter kendi başlangıç indisine yazılır, sonra o indis bir artırılır.

- İlk `d` 6. göze yazılır. Sonraki `d` artık 7. göze gelir.
- İlk `a` 0. göze yazılır. Sonraki `a` 1. göze gelir.

Sonuç:

```
tmp:    a  a  b  b  b  c  d  d  e  f  f  f
```

4. Geri kopyalama. `tmp` dizisi `a` dizisine kopyalanır.

```c
for (i = 0; i < N; i++)          /* N: a dizisinin eleman sayısı */
    count[(a[i] - 'a') + 1]++;

for (r = 0; r < R; r++)
    count[r + 1] += count[r];

for (i = 0; i < N; i++) {
    tmp[count[a[i] - 'a']] = a[i];
    count[a[i] - 'a']++;
}

for (i = 0; i < N; i++)
    a[i] = tmp[i];
```

`count` dizisini sıfırlamanın yolları:

```c
int count[7] = {0};                          /* tanımda sıfırlama (R + 1 = 7 göz) */
int *count = calloc(R + 1, sizeof(int));     /* calloc belleği sıfırlar */
for (r = 0; r <= R; r++) count[r] = 0;       /* döngüyle sıfırlama */
```

- `malloc` belleği sıfırlamaz. `calloc` sıfırlar.

`tmp` Yerine İşaretçi Değiştirme

Bir öğrenci, değerleri geri kopyalamak yerine `a` ve `tmp` işaretçilerinin yer değiştirip değiştiremeyeceğini sordu.

- Diziler statik tanımlandıysa bu yapılamaz.
- Diziler `malloc` ile ayrıldıysa yapılabilir. İşaretçi `tmp`'nin başlangıcını gösterir, eski `a` alanı `free` ile bırakılır.
- Hoca bunun doğru olduğunu ama işaretçilerle çalışırken çok dikkat edilmesi gerektiğini söyledi.
- Fonksiyon içinde yerel tanımlanan dizinin yeri fonksiyondan çıkınca bırakılır.

LSD Dizgi Sıralama

Key-indexed counting, dizgilerin her karakter konumu için sağdan sola uygulanır. Örnek: 3 karakterli 12 dizgi.

İndis	Başlangıç	3. karaktere göre	2. karaktere göre	1. karaktere göre
0	dab	dab	dab	ace
1	add	cab	cab	add
2	cab	ebb	fad	bad
3	fad	add	bad	bed
4	fee	fad	dad	bee
5	bad	bad	ebb	cab
6	dad	dad	ace	dab
7	bee	fed	add	dad
8	fed	bed	fed	ebb
9	bed	fee	bed	fad
10	ebb	bee	fee	fed
11	ace	ace	bee	fee

- **3. karakter adımında** sayılar: `b` 3, `d` 6, `e` 3. Başlangıçlar: `b` 0, `d` 3, `e` 9.
- **2. karakter adımında** sayılar: `a` 5, `b` 1, `c` 1, `d` 1, `e` 4. Başlangıçlar: `a` 0, `b` 5, `c` 6, `d` 7, `e` 8.
- **1. karakter adımında** sayılar: `a` 2, `b` 3, `c` 1, `d` 2, `e` 1, `f` 3. Başlangıçlar: `a` 0, `b` 2, `c` 5, `d` 6, `e` 8, `f` 9.

Kararlılık Neden Önemli?

- 2. karakter adımında `cab` ve `fad`'in ikinci karakterleri aynı (`a`). Bir önceki adımda `cab` üstte, `fad` altta idi (`b < d`). Kararlı sıralama bu sırayı korur.
- `fad` yukarı alınsaydı son karaktere göre yapılan sıralama boşa giderdi.
- `ebb` ile `bad` karşılaştırmasında ikinci karakterler farklıdır (`b` ve `a`). Burada yer değişmesi doğaldır: `bad` yukarı çıkar.
- Hocanın isim örneği: "Elif Karslıgil" ve "Elif Sabun" önce soyada göre sıralanır. Sonra ada göre sıralanırken adlar eşit olduğu için soyad sırası korunmalıdır. Ad farklıysa (örneğin "Ayşe Sabun") yer değişmesi sorun değildir.

LSD Sıralama Kodu (Tahtadaki Hali)

```
void LSDSort(char a[], int w, int R)   /* w: karakter sayısı, R: sembol sayısı */
{
    int i, r, d;
    char *tmp;
    /* tmp için malloc */
    for (d = w - 1; d >= 0; d--) {
        for (r = 0; r <= R; r++)
            count[r] = 0;
        for (i = 0; i < w; i++)
```

```
            count[a[i][d] - 'a' + 1]++;
        for (i = 0, r = 0; r < R; r++)
            count[r + 1] += count[r];
        for (i = 0; i < w; i++) {
            adr = a[i][d] - 'a';
            strcpy(tmp[i], a[i]);
            count[a[i] - 'a']++;
        }
        for (i = 0; i < w; i++)
            strcpy(a[i], tmp[i]);
    }
}
```

Hoca bu kodu akıldan yazdığını söyledi; kodda hatalar var. Düzeltilmiş hali "Ders sonrası notlar" bölümünde.

Vize Sorularının Çözümü

- **Soru 1 (yığın).** n'in 2'ye göre modu yığına itilir (push), n ikiye bölünür. Sonra yığın boşaltılarak yazdırılır. Program sayının ikili karşılığını yazar.
  - $n = 5$: kalanlar 1, 0, 1 itilir; pop sırası 1, 0, 1 → `101`.
  - $n = 20$ için çıktı `10100`.
- **Soru 2 (kuyruk ve yığın).** Kuyrukta 5, 2, 1, 4 var. Kuyruktan çekilip yığına itilir, yığından çekilip kuyruğa konur. Çıktı: `4 1 2 5`.
- **Soru 3 (bağlı liste).** Son düğüm olmadığı bilinen `x` için `x->next = x->next->next`. Doğru cevap: `x->next`'in eskiden gösterdiği düğüme artık kimse erişemez (bellekte kalır, `free` edilmelidir). "`x->next`, `x->next->next`'i gösterir" demek doğru cevap değildir; soru zaten bunu yapınca ne olduğunu soruyor.
- **Soru 4 (yığın ağacı, heap).** Maks-yığından en büyük eleman (38) silinir. Son eleman 19 köke gelir ve aşağı taşınır (heapify).
  - 37 ile 35 karşılaştırılır; büyük olan 37, 19 ile yer değiştirir.
  - 34 ile 36 karşılaştırılır; 36, 19 ile yer değiştirir.
  - 14 ile 17 karşılaştırılır; 17 < 19 olduğu için durulur.
  - Cevap: 14, 17, 19, 34, 35, 36, 37. 19 her adımda karşılaştırmaya girer.
- **Soru 5 (özyineleme).** Taban koşulu (base condition) yoksa sonsuz özyineleme olur; durma şartı yoktur.
- **Kuyruk karmaşıklığı sorusu.** Tek bağlı listeyle kurulan kuyrukta yalnız ilk elemanın adresi tutuluyor, son elemanın adresi tutulmuyor. En kötü durumda `enqueue` $O(n)$ (sona gitmek gerekir), `dequeue` $O(1)$.
- **Sıralama sorusu.** Bütün elemanları aynı olan n elemanlı bir dizi selection sort ile $O(n^2)$, insertion sort ile $O(n)$ sürede sıralanır. Selection sort her durumda bütün diziyi tarar; insertion sort karşılaştırma sonucuna göre erken durur.
- **Dairesel liste sorusu.** Dairesel bağlı listenin önüne eleman eklemenin maliyeti en kötü durumda $O(1)$'dir.
- **BST ekleme sorusu.** İkili arama ağacına eleman eklemenin en kötü durumu $O(n)$'dir. Ağaç tamamen dengesizse (tek kol halinde) n düğüm gezilir. Dengeli ağaçta $O(\log n)$ olur.
- **Yığın çıktısı soruları.** `push` artan sırada, `pop` ters sırada yapılıyor. `1 3 5 2 4` çıktısı oluşamaz. `1 3 5 4 2` oluşabilir.
- **Soru 12 (en çok hata yapılan soru).** Sırasız A (N eleman) ve sıralı B (M eleman) var; $N \gg M$. Ortak elemanlar: A'daki her eleman için B'de ikili arama (binary search) yapılır. Toplam $O(N \log M)$.
  - Çoğu öğrenci önce A'yı sıralamayı önerdi. Bu tek başına $O(N \log N)$ tutar ve daha pahalıdır.
- **Soru 13 ve devamı (harflerle maskelenmiş BST).** Ağaç: kök A; A'nın çocukları B ve C; B'nin çocukları D ve E; C'nin çocukları F ve G; D'nin sağında H, E'nin sağında I, F'nin solunda J, G'nin çocukları K ve L, L'nin sağında M.
  - A'dan büyük ilk değer: sağ alt ağacın en soldaki düğümü, yani **J** (C → F → J).
  - B'den büyük ilk değer: B'nin sağ alt ağacının en soldaki düğümü, yani **E**.
  - C'den büyük ilk değer: **K** (G'nin sol çocuğu).
  - Ağacın yüksekliği **4** (A → C → G → L → M).

- Yüksekliği bozmadan eklenebilecek eleman sayısı: yüksekliği 4 olan tam ağaçta $2^5 - 1 = 31$ düğüm vardır; mevcut 13 düğümle **18** eleman daha eklenebilir.

Programlama Sorusu (Arama Motoru Bağlı Listesi)

Kurallar:

- Sayfaya ilk kez bakılıyorsa kod numarası listenin başına eklenir, sayacı 1 olur.
- Ekleme sonrası listedeki eleman sayısı N'yi aşarsa en sondaki sayfa silinir.
- Sayfaya daha önce bakıldıysa sayacı artırılır.
- Sayaç K değerine ulaşırsa sayfa listenin başına alınır ve sayacı sıfırlanır.

Doğru tasarım: listeyi bir kez gez. Bu gezinti sırasında elemanı ara, eleman sayısını say ve son düğümün öncesini tut. Bulursan sayacı artır; bulamazsan başa ekle ve gerekirse sonu sil.

Yanlış ya da verimsiz tasarımlar:

- Elemanı önce başa eklemek, sonra listede arayıp bulursa sayacı artırıp baştakini silmek.
- Elemanı aramak için sona kadar gidip, ekledikten sonra eleman sayısını bulmak için listeyi baştan bir daha saymak.
- Silinecek son düğümün öncesini bulmak için üçüncü kez gezmek.
- N sınırını hiç kontrol etmeden eklemeye devam etmek.
- Elemanın listede olup olmadığını kontrol etmeden her geleni başa eklemek (aynı eleman iki kez bulunur).

Ders sonrası notlar

- **LSD kodu: döngü sınırı.** Hoca hem dizgi uzunluğu hem de dizgi sayısı için `w` kullandı. Dış döngü `d = W - 1`'den 0'a karakter konumlarını, iç döngüler `i = 0`'dan `N`'ye dizgileri gezmelidir. `N` dizgi sayısıdır, `W` dizgi uzunluğudur.
- **LSD kodu: dağıtma adımı.** Tahtada `strcpy(tmp[i], a[i])` ve `count[a[i] - 'a']++` yazıldı. Doğrusu dizgiyi hesaplanan adrese koyup o adresi artırmaktır:

```
for (i = 0; i < N; i++) {
    adr = a[i][d] - 'a';
    strcpy(tmp[count[adr]], a[i]);
    count[adr]++;
}
for (i = 0; i < N; i++)
    strcpy(a[i], tmp[i]);
```

- **LSD kodu: diğer yazım hataları.** `for (i = 0; r < R; r++)` döngüsü `for (r = 0; r < R; r++)` olmalı. Parametre `char a[]` değil `char a[][W + 1]` veya `char **a` olmalı, çünkü `a[i][d]` iki boyutlu erişimdir. `count` dizisi $R + 1$ elemanlı tanımlanmalıdır.
- **Sıralama algoritmalarının ortalama durumu.** Hoca "average case'lerin hepsi aynı" dedi. Merge sort, heap sort ve quicksort'un ortalama durumu $O(n \log n)$'dir. Selection, bubble ve insertion sort'un ortalama durumu ise $O(n^2)$'dir.
- **BST'ye ekleme.** Hoca bir yerde "en kötü durumda $O(1)$" dedi, hemen ardından dengesiz ağaç üzerinden $O(n)$ olduğunu açıkladı. Doğru cevap $O(n)$'dir.
- **Dairesel listeye başa ekleme.** $O(1)$ ancak son düğümü gösteren bir işaretçi tutuluyorsa geçerlidir. Yalnız baş işaretçisi tutuluyorsa son düğümün `next` alanını güncellemek için sona gitmek gerekir ve maliyet $O(n)$ olur.
- **Key-indexed counting örneğinde sayma.** Hoca sayarken birkaç kez karıştırdı (örneğin F'yi 9 yerine 10'dan başlattı, son adımda D'yi 7'den başlattı). Doğru değerler yukarıdaki tablolardadır: ilk örnekte F 9'dan, LSD'nin son adımında D 6'dan başlar.
- **Soru 12'deki alternatif.** B'nin elemanları bir özet tablosuna (hash table) konursa ortak elemanlar ortalama $O(N + M)$ sürede bulunabilir. Ders kapsamında beklenen cevap ikili aramadır.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.